الكلية متعددة التخصصات - ورزازات
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

Classes et Objets

Pr. OMARI Kamal

2025/2026

▪**Paradigme de programmation:** est une **façon d'organiser le code** pour résoudre un problème. Chaque paradigme propose une vision différente:

Comment représenter les données,

Comment structurer les instructions,

Comment décomposer et résoudre les problèmes.

➢Parmi les paradigmes de programmation, on trouve:

Programmation procédurale,

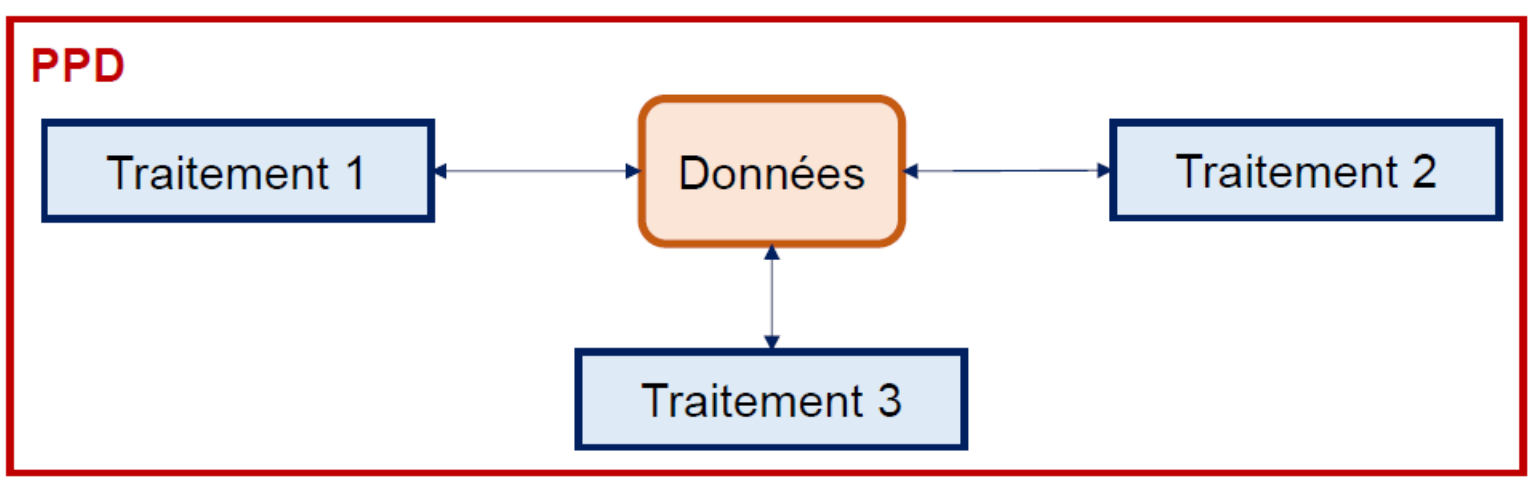
Programmation orientée objet.

▪ Programmation procédurale

➢ Le programme est divisé en petites parties appelées procédures /fonctions qui effectuent des traitements sur des données.

➢ Il n'existe pas un lien direct entre les données et les traitements.

- Les données et les traitements sont séparés.
- Les données à traiter sont passées en arguments aux procédures

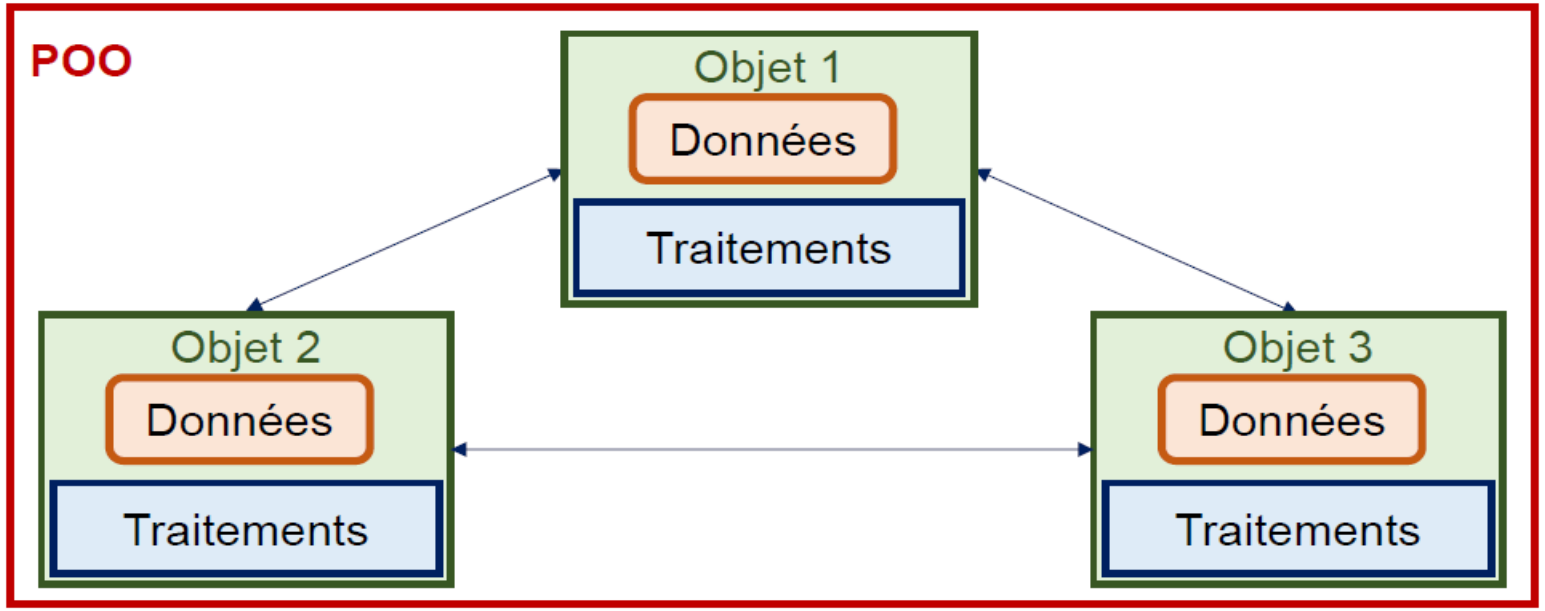


- **Programmation Orientée Objet**

➢Le programme est composé de plusieurs objets qui contiennent:

- des données « internes »,
- des traitement qui manipulent ces données internes ou d'autres données.

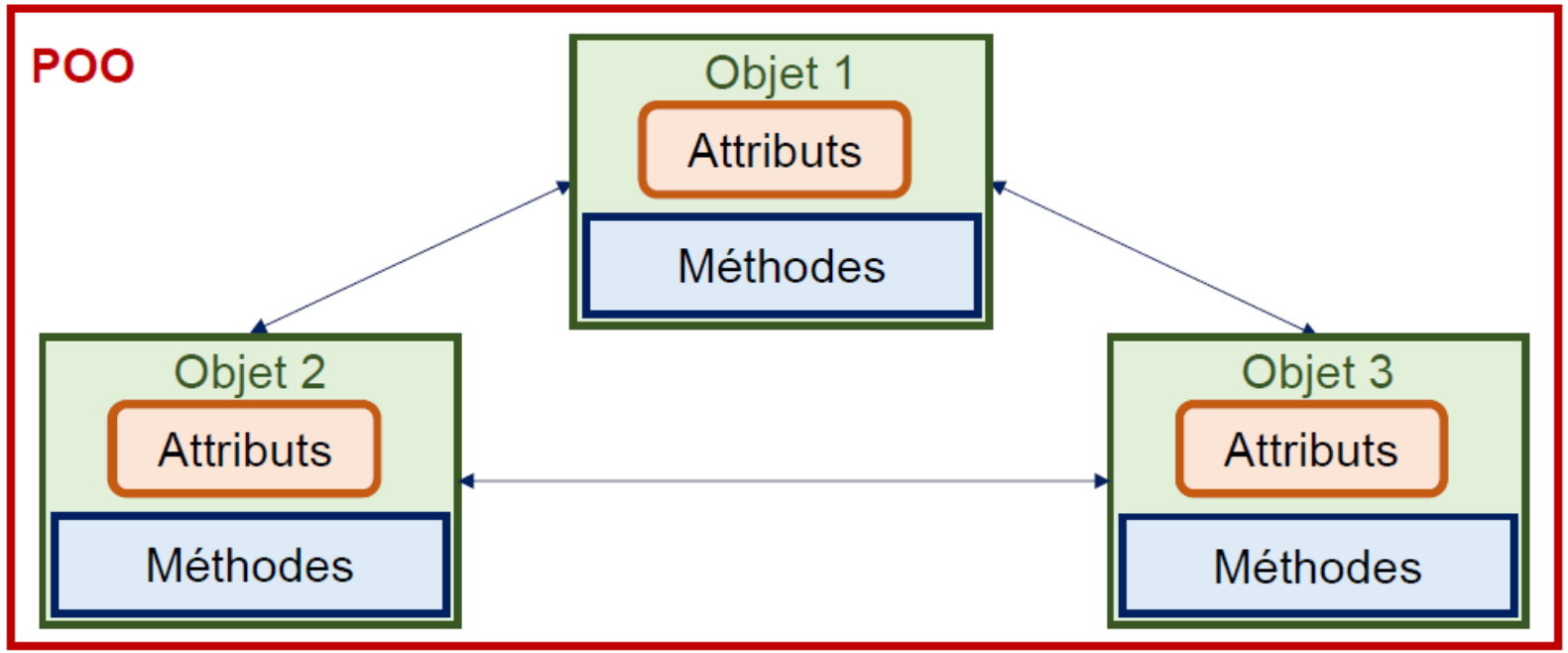
➢Les données et leurs traitements sont regroupés dans une seule entité: **objet**.



- **Programmation Orientée Objet**

➢Les données d'un objet sont appelés ses attributs.

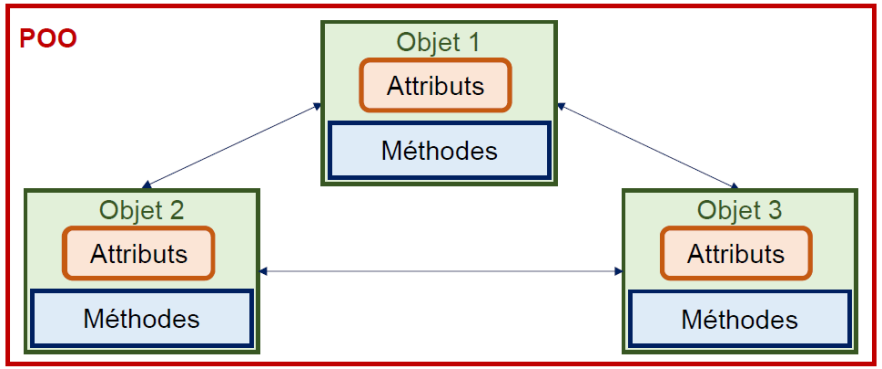
➢Les traitements d'un objet sont appelés ses méthodes.



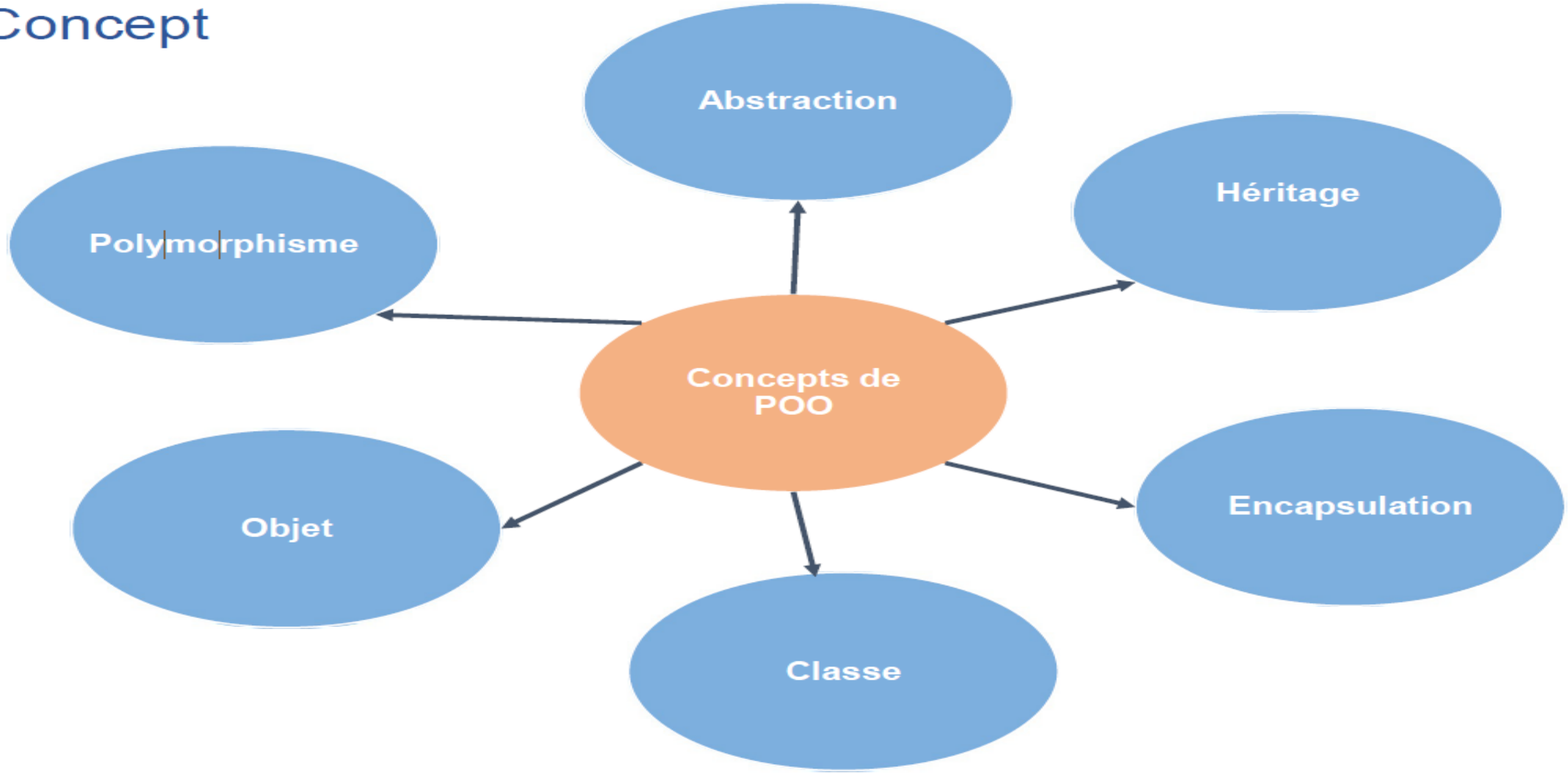
Programmation Orientée Objet

➢La plupart des langages de programmation populaires aujourd'hui sont basés sur l'approche objets. Mais les objets ne sont pas uniquement utilisés pour la programmation...

- ❑Bases de données objets(Oracle)
- ❑ Méthodes de conception objets (UML)



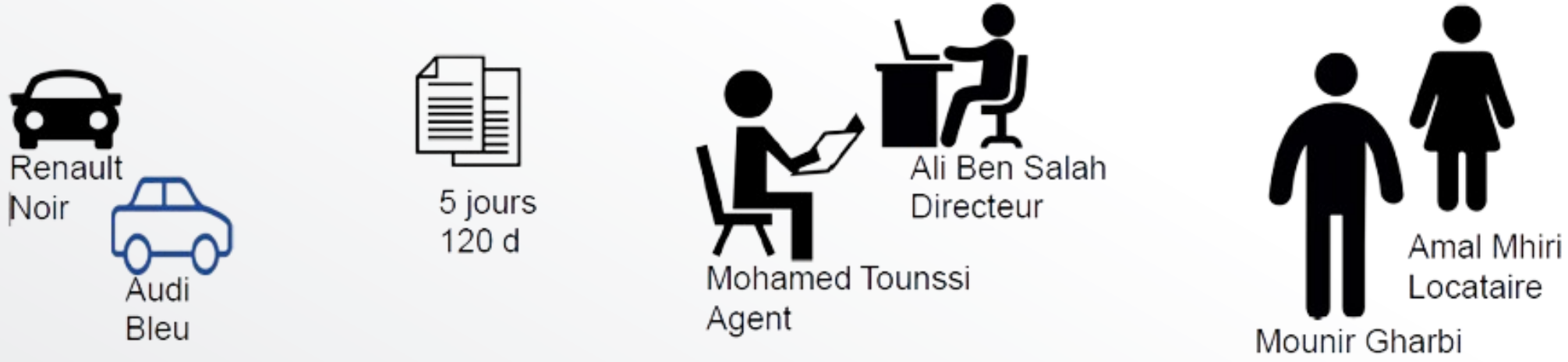
➢ Concept


Abstraction
Héritage
Polymorphisme
Concepts de POO
Objet
Encapsulation
Classe

Notion d'objet

- Le concept d'objet est utilisé pour modéliser informatiquement des objets concrets de la vie courante (véhicule, personne, etc.) ou des objets conceptuels (date, etc.).

Exemple: Gestion de location de véhicule



Notion d'objet

- Un **objet** est une entité informatique comprenant:
  - des **données membres** (ou champs/attributs)
  - des **fonctions membres** (ou méthodes)

- Un **objet** possède un état:

  L'état est donné par la valeur de ses attributs à un instant donné.
  Il peut varier au cours du temps.

- Un **objet** possède une identité:

  qui permet de le distinguer des autres objets indépendamment de son état,
  qui est construite grâce à un identifiant

Notion de classe

- Une classe est la description d'une famille d'objets ayant une même structure et un même comportement.

Notion de classe

- Une **classe** est un type de données défini par l'utilisateur,

- Une **classe** est comme un **modèle** pour un **objet**.

- Une **classe d'un objet** décrira donc:

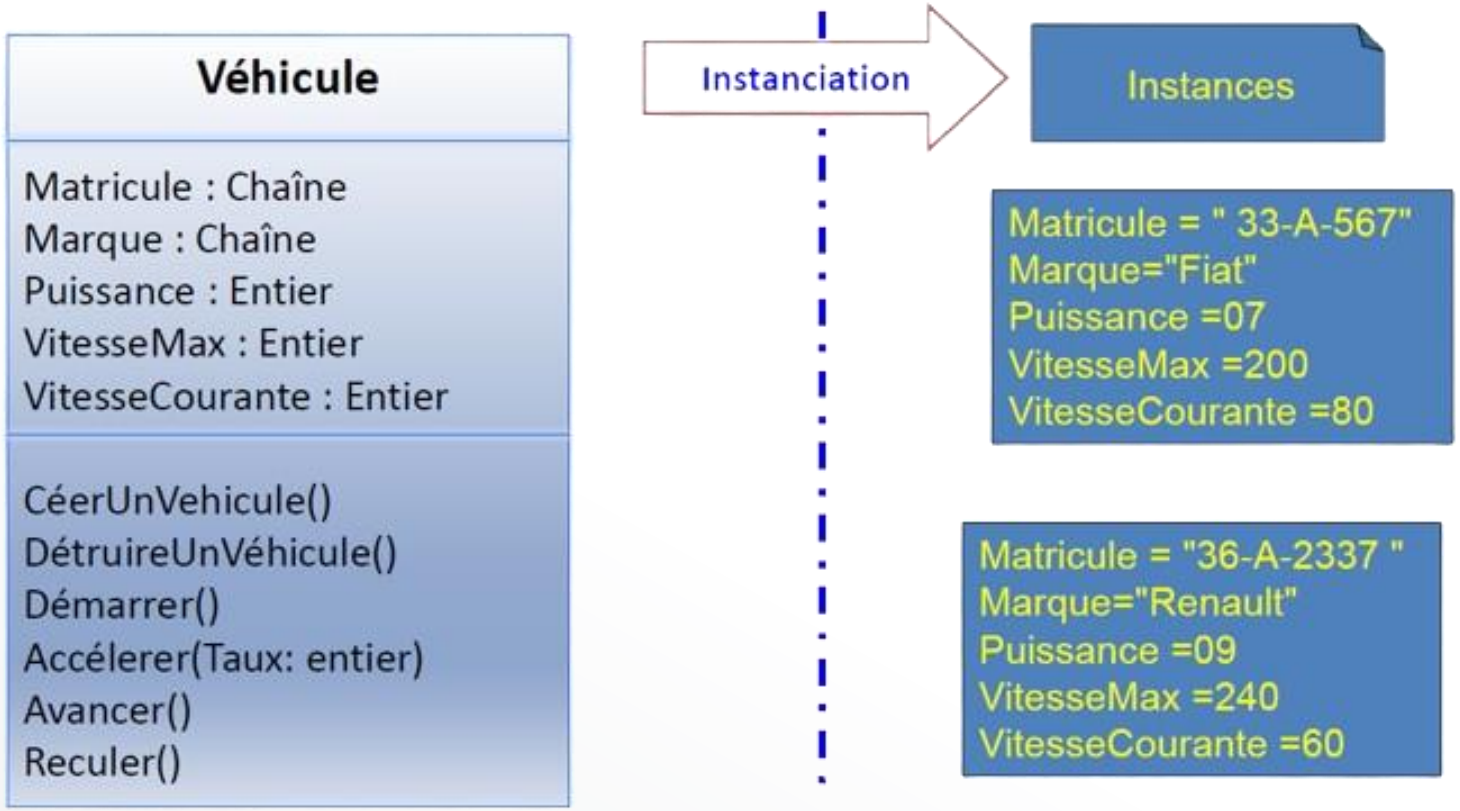
  -Des données membres.

  -Des méthodes membres qui seront des fonctions

- Les membres d'une classe sont accessibles et utilisables en créant une **instance de cette classe**.

Objet et classe: Instanciation

- Pour désigner un objet de la classe, on dit aussi une *instance.*
- L'instanciation est le mécanisme de **création d'objets à partir d'une classe.**
- Une classe peut être réutilisée pour instancier plusieurs objets.



Syntaxe de définition d'une classe

```
modificateurs class nom_de_classe
{
    ...
}
```

- Exemple:

```
public class Point {
    // 1. Attributs (coordonnées du point)
    private int x;
    private int y;
    // 2. Constructeur (pour initialiser le point)
    public Point(int abs, int ord) {
        x = abs;
        y = ord;
    }
    // 3. Méthode pour déplacer le point
    public void deplacer(int dx, int dy) {
        x = x + dx;
        y = y + dy;
    }}
```

Modificateurs de la classe

Modificateur	Rôle
abstract	La classe contient une ou des méthodes abstraites, qui n'ont pas de définition explicite. Une classe déclarée abstract ne peut pas être instanciée : il faut définir une classe qui hérite de cette classe et qui implémente les méthodes nécessaires pour ne plus être abstraite.
final	La classe ne peut pas être modifiée, sa redéfinition grâce à l'héritage est interdite. Les classes déclarées final ne peuvent donc pas avoir de classes filles.
private	La classe n'est accessible qu'à partir de la classe où elle est définie (les classes internes).
public	La classe est accessible partout

Objet

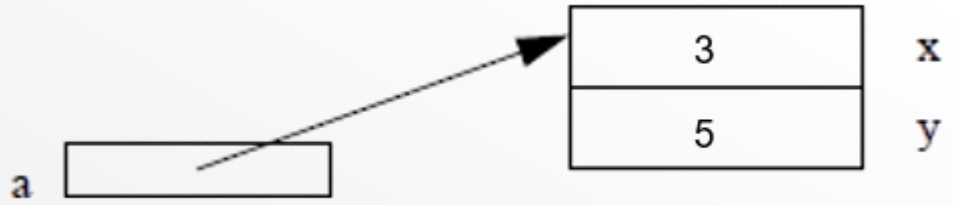
- La classe est la description d'un objet.
- Un objet est une instance d'une classe.
- L'opérateur new crée une instance d'une classe en faisant appel à une méthode particulière appelée constructeur, et renvoie une référence sur l'objet instancié.
- Pour chaque instance d'une classe, le code est le même, seules les données sont différentes à chaque objet.

La création d'un Objet

- Il est nécessaire de déclarer une variable ayant le type de l'objet désiré.
  - La déclaration est de la forme: <nomClasse> <variable>;
  - Exemple: Point a;
  - Contrairement à la déclaration d'une variable d'un type primitif (comme int n ;), **elle ne réserve pas d'emplacement pour un objet** de type Point, mais seulement **un emplacement pour une référence** à un objet de type Point.
- L'emplacement pour l'objet proprement dit sera alloué sur une demande explicite du programme, en faisant appel à l'opérateur unaire nommé **new.**

La création d'un Objet

- L'opérateur new se charge de créer une instance de la classe et de l'associer à la variable (place sa référence dans variable):
  - <variable> = new <nomClasse>(<paramètres>) ;
  - Exemple: **Point a = new Point(3,5);** // crée un objet de type Point et place sa référence dans a.



  - Les valeur de x et y sont initialisées ici à 3 et 5.

Initialisation d'un objet

- Les attributs d'un objet sont toujours initialisés par défaut.
- Dès qu'un objet est créé, et avant l'appel du constructeur, ses attributs sont initialisés à une valeur par défaut ainsi définie:

Type du champ	Valeur par défaut	Explication
boolean	false	Par défaut, faux.
char	'\u0000' (caractère vide)	Représente un caractère nul
byte, short, int, long	0	Tous les entiers valent 0.
float, double	0.0	Tous les nombres réels valent 0.0.
objet (ex : String, Point, etc.)	null	Aucune référence : l'attribut ne pointe vers aucun objet.

Encapsulation

- L’accès direct aux variables d'un objet est possible en JAVA
- **MAIS** ... n'est pas recommandé car contraire au principe d'encapsulation
  - les données d'un objet doivent être privées (c'est à dire protégées et accessibles(et surtout modifiables) qu'au travers de méthodes prévues à cet effet).
- Plusieurs niveaux de visibilité peuvent être définis en précédant la déclaration de chaque attribut ou méthode d’un modificateur (**private, public, protected**)

Les modificateurs d'accès

	public	protected	private
Attributs	Attribut accessible depuis code de n'importe quelle classe.	Attribut accessible dans les classes filles et classes du même package.	Attribut accessible uniquement dans le code de la classe qui le définit
Méthode	Méthode pouvant être invoquée depuis n'importe quelle classe.	Méthode pouvant être invoquée dans les classes filles et classes du même package.	L'usage de la méthode est réservé aux méthodes de la même classe.

Sans modificateur, une entité (méthode ou attribut) est visible par toutes les classes se trouvant dans **le même package**.

Constructeurs

- Un **constructeur** est une **méthode spéciale** d'une classe qui est **appelée automatiquement** lors de la **création d'un objet**.
- Porte **le même nom que la classe**.
- **Ne retourne aucune valeur** (pas même void).
- Peut être **surchargé** (plusieurs constructeurs avec des paramètres différents).
- Est **appelé automatiquement** par **new.**

Constructeur Par défaut

- C'est un constructeur **sans paramètre**.
- Il est **créé automatiquement** par le compilateur **si aucun autre constructeur n'est défini**.

```
public class Point {
    // Attributs (coordonnées du point)
    private int x;
    private int y;
   // Constructeur par défaut
    public Point() {
        x = 0;
        y = 0;
    }
}
➢  Point p = new Point(); // appel automatique du constructeur par
  défaut
```

Constructeur paramétré

- Il sert à **initialiser les attributs** de l'objet **avec des valeurs données** lors de sa création.

```
public class Point
{
    // Attributs (coordonnées du point)
    private int x;
    private int y;
   // Constructeur paramétré
    public Point(int abs, int ord) {
        x = abs;
        y = ord;
    }
}
➢ Point p = new Point(3, 5); // initialise x=3, y=5
```

Constructeur de copie (1)

- Le constructeur de copie permet de créer un nouvel objet à partir d'un autre objet du même type. Autrement dit, **il duplique les attributs d'un objet existant.**

```
public class Point
{
    private int x, y;
   // Constructeur paramétré
    public Point(int abs, int ord) {
        x = abs;
        y = ord;
    }
   public Point(Point p) {
        this.x = p.x;
        this.y = p.y;
    }
}
```

Constructeur de copie (2)

- Utilisation:

```
public class TestPoint {
    public static void main(String[] args) {
        Point p1 = new Point(3, 4);
        Point p2 = new Point(p1);     // création par copie

        p1.afficher();  // (3, 4)
        p2.afficher();  // (3, 4)
    }
}
```

Constructeur de copie (2)

- Utilisation:

```
public class TestPoint {
    public static void main(String[] args) {
        Point p1 = new Point(3, 4);
        Point p2 = new Point(p1);     // création par copie

        p1.afficher();  // (3, 4)
        p2.afficher();  // (3, 4)
    }
}
```

Destruction d’objets (1)

- Java n’a pas prévu la notion de destructeur telle quelle en existe en C++.
- La destruction des références d’objets se fait automatiquement grâce à un processus appelé "ramasse miettes" (ou **garbage collector**) qui s’occupe de collecter les objets qui ne sont plus référenciés.

Destruction d'objets (2)

- Exemple **(C++ vs JAVA)**

```
class Personne {
private:
   string nom;
public:
   Personne(string n) {
      nom = n; }
   ~Personne() {    // Destructeur
      cout << "Destructeur : " << nom << endl;
   } };
int main() {
   Personne* p = new Personne("Ali");
   cout << "Programme en cours..." << endl;
   delete p;
 cout << "Fin du programme" << endl; }
```

```
class Personne {
   private String nom;
   public Personne(String n) {
      nom = n;
      System.out.println("Constructeur : " + nom + "
(objet créé)");
   }
   public void afficher() {
      System.out.println("Bonjour, je suis " + nom);
   } }
public class Test {
   public static void main(String[] args) {
      Personne p = new Personne("Ali");
      p.afficher();
      p = null;
      System.out.println("Fin du programme");}}
```

Variables d’instance

- Une variable d'instance nécessite simplement une déclaration de la variable dans le corps de la classe.

```
public class MaClasse {
 private int valeur1, valeur2 ;
 protected int valeur3 ;
 private Point valeur4 ;
}
```

- Chaque objet de la classe possède sa propre copie de ces variables.
- Cela permet de différencier un objet d’un autre.

Variable de la classe

- La variable de la classe est définie avec le mot clé static.
- Elle appartient à la classe elle-même, pas à chaque objet.
- Elle est partagé par tous les objets d'une même classe.
- Elle garde une seule et unique valeur, commune à tous les objets.

```
public class Point{
private int x ; private int y ;
  static int compteur = 0;
  public Point(int x,int y){
    this.x = x ;
    this.y = y ;
    compteur++;
  }
}
```

```
...
Point p1 = new Point(3,8);
Point p2 = new Point(7,10);
Point p3 = new Point(10,20);
System.out.println("Il y a " +
Point.compteur + " points");
// l'instruction affichera : Il y a
3 points
```

Méthode de la classe (1)

- Une méthode de classe est une méthode qui ne nécessite pas la création d'un objet, elle peut être appelée sans instance de la classe.
- On utilise là aussi le mot réservé static.
- Une méthode de classe **ne peut accéder qu'aux attributs de la classe déclarés comme static**.
- L'appel d'une méthode de classe ne se fait **pas sur un objet**, mais **sur une classe**.
  - Exemple : Math.cos(3.14);

Méthode de la classe (2)

- Exemple

```
public class MathCls {
 private final static double PI = 3.14 ;
 public static double getPI() {
     return PI;}
 public static double carre(double x) {
    return x*x;}
 public static double demi(double x) {
   return x / 2; }
}
```

```
double i = MathCls.carre(5);
double x = MathCls.getPI();
```

Méthodes accesseurs / modificateurs (1)

- L'encapsulation permet de sécuriser l'accès aux données d'une classe.
- Ainsi, les données déclarées private à l'intérieur d'une classe ne peuvent être accédées et modifiées que par des méthodes définies dans la même classe.
- Par convention, les accesseurs commencent par **get** et les modificateurs commencent par **set**.

Méthodes accesseurs / modificateurs (2)

```
public class Point{
  private int x, y ;
  public Point (int x, int y){
  this.x = x ;
  this.y = y;}

 public int getX(){ return x; }
 public int getY(){ return y;}

 Public void setX(int x){
   this.x = x;
 }
 public void setY(int y){
   this.y = y;
 }

}
```

Accesseurs

Modificateurs

L’autoréférence this(1)

Le mot-clé **this** représente **l’objet courant** sur lequel une méthode ou un constructeur est exécuté.

Parmi les cas d’utilisation de this:

1. Distinguer un attribut d’un paramètre.
2. Retourner l’objet courant.

L’autoréférence this(2)

1. **Distinguer un attribut d’un paramètre.**

```
public class Point {
    private int x, y;
    // Constructeur
    public Point(int x, int y) {
        this.x = x;  // this.x: attribut de l’objet
        this.y = y;  // "y" → paramètre
    }
```

L’autoréférence this(4)

2. Retourner l’objet courant

```
public class Point {
    private int x, y;
    public Point(int a, int b) {
        x = a;
        y = b;
    }
    public Point afficher() {
        System.out.println("(" + x + ", " + y + ")");
        return this; // renvoie l'objet courant
    }
    public Point deplacer() {
        x++;
        y++;
        return this;
    }
}
```

```
public class TestPoint {
    public static void main(String[] args) {
        Point p = new Point(2, 3);
        p.afficher().deplacer().afficher();
    }
}
```

Surcharge des fonctions (1)

La surcharge (ou ***Overloading***) consiste à définir plusieurs fonctions membres portant **le même nom**, mais ayant:

- des **listes de paramètres différentes** (nombre ou type),
  - Le compilateur choisit la bonne fonction en fonction des arguments passés.
  - La surcharge n'est possible que si les fonctions se distinguent par:

    -le nombre de paramètres,

    -ou le type des paramètres,

    -ou les deux à la fois.
- La valeur de retour seule ne permet pas de surcharger.

Surcharge des fonctions (2)

Remarque: Puisque les constructeurs sont aussi des **méthodes spéciales**, ils peuvent donc être **surchargés** exactement comme les autres méthodes.

Le compilateur choisit **automatiquement** le bon constructeur selon: le **nombre**, le **type** des paramètres passés.

Exemple:

```
public class Point {
   private int x, y;
public Point(int a, int b) {
      x = a;
      y = b;}
public Point(int a) {
      x = a;
      y = a;}
public void afficher() {
      System.out.println("(" + x + ", " + y + ")");}
public void afficher(String msg) {
      System.out.println(msg + " (" + x + ", " + y + ")");}
}
```

```
public class TestPoint {
   public static void main(String[] args) {
      Point p1 = new Point(3, 5);
      Point p2 = new Point(4);
      p1.afficher();
      p2.afficher("Coordonnées du point :");
}
}
```

Exercice d’application

1. Créez une classe Voiture avec les **attributs**: marque: (String), vitesse(int) et nbVoitures: représentant le nombre total de voitures créées.

2. Créez le **constructeur par défaut qui** initialise marque = "Inconnue" et vitesse = 0.

3. Créez **le constructeur paramétré:** initialise marque et vitesse avec les valeurs données.

➢ À chaque création d’objet, incrémentez nbVoitures.

4. Créez les **méthodes d’accès et de modification**: getMarque(), getVitesse(), setMarque(String), setVitesse(int).

5. Ajoutez une **méthode d’affichage** qui affiche la marque et la vitesse d’une voiture.

6. Ajoutez une **méthode surchargée accelerer()**:

- Version sans paramètre → augmente la vitesse de 10 km/h,
- Version avec un paramètre entier → augmente la vitesse du nombre de km/h passé.

7. Ajoutez une **méthode** qui affiche le **nombre total d’objets Voiture créés**.

➢ **Dans une classe TestVoiture**, écrivez le programme principal:

1. Créez trois objets: v1, v2 et v3, en initialisant les attributs de v2 et v3.
2. Affichez les informations de chaque voiture.
3. Changez la vitesse de v1.
4. Appelez accelerer() (avec et sans paramètre) sur v2.
5. Affichez le nombre total de voitures créées.

Solution (1)

```
public class Voiture {
   // Attributs d'instance
   private String marque;
   private int vitesse;

   // Attribut de classe (partagé)
   private static int nbVoitures = 0;

   // 1) Constructeur par défaut
   public Voiture() {
      this.marque = "Inconnue";
      this.vitesse = 0;
      nbVoitures++; // compter chaque objet créé}
 // 2) Constructeur paramétré
   public Voiture(String marque, int vitesse) {
      this.marque = marque;
      this.vitesse = vitesse;
      nbVoitures++; }
```

```
 // 3) Accesseurs / mutateurs
   public String getMarque() { return marque; }
   public int getVitesse() { return vitesse; }
   public void setMarque(String marque) {
            this.marque = marque; }
   public void setVitesse(int vitesse)   {
            this.vitesse = vitesse; }
  // 4) Affichage
   public void afficher() {
      System.out.println("Voiture[" + marque + "] - vitesse = " +
vitesse + " km/h") }
// 5) Surcharge de accelerer()
   public void accelerer() {
            this.vitesse += 10;
   }
   public void accelerer(int v) {
            this.vitesse += v; }
 // 6) Méthode de classe pour afficher le total
   public static void afficherNbVoitures() {
      System.out.println("Nombre total de voitures créées : " +
nbVoitures);}
```

Solution (2)

```
public class TestVoiture {
   public static void main(String[] args) {
      // Créez trois objets (v2 et v3 initialisés avec
des paramètres)
      Voiture v1 = new Voiture();
      Voiture v2 = new Voiture("Peugeot", 50);
      Voiture v3 = new Voiture("Renault", 90);
      // Afficher les infos de chaque voiture
      v1.afficher();
      v2.afficher();
      v3.afficher();
      // Changer la vitesse de v1
      v1.setVitesse(30);
      v1.afficher();
```

```
// Appeler accelerer() (avec et sans paramètre) sur
v2
      v2.accelerer();        // +10
      v2.accelerer(25);      // +25
      v2.afficher();
// Afficher le nombre total de voitures créées
      Voiture.afficherNbVoitures();   // méthode
statique appelée via la classe
   }
}
```